

Stoquify Internal Activation Gate Blocker Closure and Next-Stage Execution Report

Date: 2026-07-24 **Execution scope:** Agent Runtime Phase 2 controlled-pilot trust gates **Repository result:** Repository-addressable assurance advanced and verified locally **Final decision:** NO-GO - READY FOR CONTROLLED REMEDIATION VERIFICATION **Internal activation:** Blocked and not attempted **Phase 3:** Not authorized

1. Executive Decision

The refined blocker-closure prompt was executed against Stoquify's current repository, local PostgreSQL database, release-control evidence, CI configuration, browser certification harness, and architecture graph.

This execution closed several material repository gaps:

- Added the governed **SUSPENDED** -> **RETIRED** lifecycle transition with tenant scope, optimistic concurrency, fresh-auth protection, RBAC, and audit evidence.
- Added terminal webhook **DEAD_LETTER** behavior, bounded exponential retry jitter, and terminal-state deduplication.
- Added a tenant-scoped protected-business-data fingerprint covering accounting, compliance, payment, inventory, purchasing, payroll, role, and user-role state.
- Bound the before/after fingerprint to the sanitized Playwright certification report.
- Expanded desktop and mobile certification to prove keyboard launch, valid evidence-link or empty-state behavior, feedback persistence, idempotent replay, and absence of horizontal overflow.
- Removed the temporary Prisma schema and generated-client workaround artifacts.
- Revalidated the complete local migration, static control, unit/component, PostgreSQL, and browser evidence chain.

The final local Command Agent package is **PILOT_CERTIFIED** and remains inactive. Its **activatedAt** field is null. Protected business data was unchanged across certification.

Stoquify is still not ready for a controlled internal pilot because the remaining blockers require real authority or production-like infrastructure:

1. Potentially exposed non-test credentials have not been proven rotated.
2. The worktree is not committed, reviewed, or certified by CI from an exact clean commit.
3. Product and security approvals are represented only by E2E identities, not real decisions.
4. The six operational responsibilities are represented only by E2E records, not an accepted real roster.
5. No production-like five-minute scheduler or managed reconciler credential has been deployed.
6. No production-like alert has been delivered, acknowledged, and escalated to a named owner.
7. The full negative and degradation browser matrix is not complete.
8. The independent enterprise release gate has not issued a final decision on the completed evidence bundle.

The correct next step is a production-like remediation-verification cycle. Activation and Phase 3 remain prohibited.

2. Revalidated Current State

Area	Current state	Classification
Canonical agent runtime	Shared Stoquify-native runtime with deterministic Command Agent	Passed locally
Business authority	Static gate finds no agent business-write path	Passed locally
Provisioning	Provisioner creates DRAFT/SHADOW definitions and rejects direct activation	Passed locally
Tenant release package	Final package is <code>PILOT_CERTIFIED</code>	Passed locally
Activation	<code>activatedAt: null</code> ; activation not attempted	Safely inactive
Approval records	Two distinct E2E approvers	Test evidence only
Owner records	Six E2E responsibility records	Test evidence only
Retirement	Guarded retirement command and audit evidence	Passed locally
Alert exhaustion	Fifth failure becomes <code>DEAD_LETTER</code>	Passed locally
Alert transport	Local environment reports <code>webhook_url_missing</code>	Blocked - deployment
Reconciliation	Local control-plane reconciliation persists findings and incidents	Passed locally
Reconciliation schedule	No deployed five-minute job or managed identity/secret evidence	Blocked - deployment
Browser happy path	Desktop and mobile pass	Passed locally
Protected-data non-mutation	Before and after hashes match	Passed locally
Negative/degradation matrix	Important cases remain unexecuted	Partial
Clean-commit CI	Current worktree is dirty; no immutable CI run	Blocked - repository/process
Real governance	No real product/security decisions or owner acknowledgement	Blocked - human authority
Credential response	Durable report is clean; real rotation evidence absent	Blocked - security authority
Enterprise release gate	Not yet rerun on a complete production-like evidence bundle	Blocked by dependencies

The architecture graph reinforces the chosen ownership model. Its Server Action Security Stack, Tenant Defence In Depth, and Enterprise RBAC Control Plane communities support the current sequence of protected action, tenant-scoped service, RBAC decision, domain-owned read adapter, evidence persistence, and audit. The agent runtime is not being treated as an alternate system of record.

3. Superseded Findings

Several findings in the earlier reports are now outdated:

Earlier finding	Current result
<code>RETIRED</code> existed only as an enum	A dedicated guarded retirement command, action permission, migration fields, and audit event now exist
Alert retries ended in generic failure	The fifth attempt now enters terminal <code>DEAD_LETTER</code>
Retry timing was deterministic	Retry delay now has bounded 80-100 percent jitter over capped exponential backoff
Duplicate queueing could reset terminal delivery state	Queue upsert now preserves terminal delivery status and attempts
Browser certification did not prove business-data non-mutation	A versioned tenant-scoped fingerprint is now certified before and after
Browser certification did not exercise feedback or replay	Desktop and mobile now persist <code>Helpful</code> feedback and return the existing run receipt on replay
Keyboard execution was unverified	The final browser run focuses the generate control and executes it with Enter

Earlier finding	Current result
Temporary Prisma workaround artifacts remained	The temporary schema and generated-client directory were removed

These closures reduce repository risk. They do not satisfy the remaining human, deployment, external-service, and clean-commit gates.

4. Final Local Evidence

4.1 Certified package

- Organization: `org_payroll_e2e_local`
- Package ID: `cmryufocd0003ma00towk2vex`
- Package state: `PILOT_CERTIFIED`
- Activation timestamp: `null`
- Release version: `e2e-ac30ee75314a-1784891703008`
- Recorded Git SHA: `ac30ee75314a0a2a2fcd6bd2ed65afb280aa0d5e`
- Manifest hash: `sha256:f6d5f1049e0baff64bb42503077f3fe801bd214bcf265289308f7db56f51c28a`
- Certification ID: `cmryup0x40001mazksmg7kxpo`

The recorded Git SHA is the current HEAD used by the local harness, but the certification remains provisional because the worktree contains uncommitted changes.

4.2 Browser evidence

- Playwright: 4 expected, 4 passed, 0 unexpected, 0 skipped, 0 flaky
- Projects: auth setup, desktop enabled pilot, mobile enabled pilot
- Sanitized report SHA-256: `3b555af2b08e744007f86790db4796c7f81d086a4a85f12f48424fabbbe4b510`
- Raw report retained: `false`
- Environment included: `false`
- Forbidden credential-bearing key matches: `0`
- Raw report path exists: `false`

The final run proves:

- Authenticated tenant-scoped access
- Read-only trust disclosure
- Keyboard execution
- Evidence-backed brief or explicit permitted empty state
- Tenant-internal link policy for every evidence link present
- Feedback persistence
- Idempotent replay receipt
- Desktop and mobile rendering
- No horizontal overflow
- No protected-business-data mutation

4.3 Protected-data fingerprint

- Version: `agent-protected-business-data-v1`

- Before: sha256:0392d9c5a9435798a360341537d14b5c29f064985b84a83df26d14366d7d1628
- After: sha256:0392d9c5a9435798a360341537d14b5c29f064985b84a83df26d14366d7d1628
- Result: unchanged

The fingerprint uses tenant-scoped, ordered, safe projections. It covers the protected state of journal entries and lines, compliance submissions, payments and payment transactions, inventory transactions, stock adjustments and transfers, purchase orders, supplier invoices, payroll runs, payroll payment batches, salary-change requests, payment-destination-change requests, roles, and user-role memberships. It does not place raw personal or secret data in the browser report.

4.4 PostgreSQL smoke

- Package ID: cmryufocd0003ma00towk2vex
- State: PILOT_CERTIFIED
- Activated: no
- Approval records: 2 E2E records from distinct identities
- Owner records: 6 E2E records
- Certification records: 1
- Audit records: 18
- Reconciliation: 10 scanned, 9 passed, 1 failed
- Blocking reason: alert transport missing
- Automatic suspension: passed with audit evidence
- Global definition after tenant suspension: ACTIVE/SHADOW
- Retirement after suspension: RETIRED with audit evidence
- Approval replay: idempotent
- Certification replay: idempotent
- Conflicting replay: RELEASE_IDEMPOTENCY_CONFLICT
- Dead-letter fixture: status DEAD_LETTER, attempt count 5, no next attempt
- Activation remained blocked by stale reconciliation

The synthetic dead-letter fixture proves persistence and worker behavior. It does not prove a real alert endpoint, owner acknowledgement, or escalation.

5. Verification Register

Verification	Result
Focused ESLint	Passed
Full TypeScript check	Passed
Prisma schema validation	Passed
Migration safety gate	8/8 checks, 33 migrations, 0 risk findings
Local PostgreSQL migration status	Database schema up to date
Agent runtime static gates	Passed
Agent prohibition gate	Passed; no business-write path in <code>services/agents</code>
Focused Jest	20 suites, 61 tests passed
Release-control PostgreSQL smoke	Passed
Final desktop/mobile Playwright	4/4 passed
Protected-data fingerprint	Passed; before equals after
Durable browser credential scan	0 forbidden-key matches
Raw browser report absence	Passed
<code>git diff --check</code>	Passed; line-ending warnings only
Full <code>verify:ci</code> from clean commit	Not run
Production-like scheduler	Not deployed
Production-like alert acknowledgement	Not run
Real credential rotation	Not evidenced
Independent enterprise release gate	Not run on final external evidence

Two intermediate browser runs failed while strengthening the scenario. The first assumed every valid digest contains a priority link; the seeded digest correctly returned an empty state. The second used a non-exact `Helpful` selector that also matched `Not helpful`. Both test assumptions were corrected. No certificate was issued from either failed run. The final corrected run passed.

6. Architecture and Trust Boundaries

The implemented user path is:

```
Command Agent panel -> protected server action -> release authorization ->
tenant/RBAC/module checks -> read-only domain adapters -> output validation/redaction
-> agent run/feedback/audit persistence
```

The implemented control path is:

```
authenticated scheduler route -> abandoned-run reconciler -> release-control reconciler
-> Workflow Assurance check/finding/incident -> webhook worker -> guarded suspension
```

The intended release path is:

```
prepare -> review -> independent approvals -> owner acceptance -> provision inactive ->
certify -> reconcile -> separate activation decision -> ACTIVE_INTERNAL
```

The agent does not own any transition in the release path. Human actors with protected permissions do.

The permanent business boundary remains:

- No direct agent Prisma business writes
- No ledger posting

- No payment execution
- No statutory filing
- No payroll or salary authority
- No stock mutation
- No close certification
- No role, permission, or entitlement changes
- No product/security self-approval

This is the correct architecture for Stoquify. It preserves the product's moat in tenant-scoped evidence, OHADA-aware domain truth, stock-to-cash-to-close continuity, and controlled execution instead of placing business truth inside an agent framework.

7. Blocker-by-Blocker Closure Matrix

Blocker	Current status	Required solution	Pass evidence	Accountable owner
Potential credential exposure	Blocked - security authority	Rotate every potentially exposed non-test credential, revoke old versions, restart dependants, verify old rejection and new success	Secret-manager version refs, timestamps, deployment restart, old-version rejection	Security owner
Dirty worktree	Blocked - repository/process	Review intended changes, create an authorized commit, run CI from exact SHA	Commit SHA, review, green CI run, immutable artifacts	Engineering/release owner
Product approval	Blocked - human authority	Real authorized product approver records decision against exact manifest	Approval ID, actor ID, rationale, validity, audit event	Product owner
Security approval	Blocked - human authority	Distinct security approver records decision after credential and threat review	Approval ID, actor ID, rationale, validity, audit event	Security owner
Operational owners	Blocked - human authority	Assign and acknowledge all six responsibilities with primary/backup coverage	Owner record IDs, runbook acceptance, validity, escalation refs	Operations lead
Five-minute reconciliation	Blocked - deployment	Deploy exact commit, managed credential, five-minute POST job, bounded concurrency and timeout	Deployment digest, schedule config, three successful windows, heartbeat	Platform/SRE
Alert delivery and acknowledgement	Blocked - external/deployment	Configure HTTPS webhook and secret, deliver test incident, acknowledge, force retry/dead-letter, prove escalation	Delivery IDs, external ref, ack event, escalation event, dead-letter evidence	SRE/security incident owner
Enabled-pilot certification	Partial	Add remaining negative/degradation/accessibility cases and rerun in CI	Clean-commit report hash and complete scenario register	QA/release lead
Enterprise release decision	Blocked by dependencies	Run skill 017 after all evidence exists	Signed go/conditional-go/no-go report	Independent release authority

8. Security and Credential Response

The durable browser runner is now designed correctly: it writes the raw Playwright JSON to a temporary path, distills a bounded report, rejects forbidden key classes, deletes the raw report, adds the protected-data assurance block, and only then hashes and records certification.

That prevents recurrence. It does not prove that previously loaded non-test credentials were never copied by endpoint indexing, backup, synchronization, or security tooling.

The security owner must perform this closure:

1. Identify each non-test credential available to the earlier local browser process without printing its value.
2. Map it to a secret-manager reference and dependent workloads.
3. Create a new secret version or credential.
4. Deploy dependent workloads using the new version.
5. Verify successful authentication and operation.
6. Revoke the old version.
7. Verify the old version fails.
8. Record owner, timestamps, references, affected environments, and verification result.
9. Invalidate related sessions or access tokens when applicable.
10. Attach the rotation record to the security approval.

The gate remains blocked until this evidence exists. A statement that the file was deleted is not sufficient.

9. Clean-Commit and CI Closure

The local package is bound to a recorded HEAD, but the working tree is dirty. It therefore cannot be the final pilot certificate.

Required procedure:

1. Review the full diff and separate unrelated user work from the intended Agent Runtime change set.
2. Confirm temporary files, auth states, raw reports, credentials, database URLs, and local traces are not included.
3. Re-run focused lint, typecheck, migration safety, static gates, unit/component tests, PostgreSQL smoke, and browser certification.
4. Create a commit only after explicit user authorization.
5. Push to a controlled branch only after explicit user authorization.
6. Run the `command-agent-enabled-pilot` CI job from the exact commit.
7. Retain the sanitized report, E2E state, test results, CI run ID, and immutable artifact link.
8. Record the deployed artifact or image digest.
9. Create a new release package bound to that exact commit and manifest.
10. Reject any certificate if the source changes after the CI run.

The final certificate must bind commit SHA, release version, manifest hash, prompt hash, schema versions, tool/skill versions, migration set, CI run ID, Playwright report hash, and expiry.

10. Governance and Ownership Closure

10.1 Approval procedure

Product and security decisions must be made by distinct real authorized users:

1. Release preparer freezes the exact manifest.
2. Product approver uses fresh authentication and `agent.release.approve.product`.
3. Security approver independently uses fresh authentication and `agent.release.approve.security`.
4. Each decision records rationale, residual-risk acceptance, evidence hash, validity, and audit event.
5. Rejection, revocation, expiry, or manifest change invalidates readiness.
6. E2E identities and dry-run records are never promoted into real approval.

10.2 Required owner roster

Responsibility	Required accountable outcome
Rollout owner	Controls scope, window, pilot allowlist, and go/no-go coordination
Rollback owner	Can suspend the package and verify user denial and data integrity
Support owner	Receives pilot incidents and maintains user communication
Pilot owner	Owns tenant acceptance, role scope, and daily validation
Security-incident owner	Owns containment, credential response, and incident escalation
On-call backup	Provides acknowledged coverage when the primary owner is unavailable

Each record requires a real directory identity, a distinct backup where required, acknowledged runbook version, coverage start/end, validity, and escalation reference.

11. Scheduler and Managed Authentication

The current provider-neutral endpoint is:

```
POST /api/internal/agents/reconcile-abandoned
Authorization: Bearer <managed value>
```

Current response behavior:

- Missing server secret: 503
- Missing or invalid bearer credential: 401
- Authorized execution: reconciles abandoned runs, reconciles release controls, dispatches alert deliveries, and returns a correlation ID

Minimum deployment design:

1. Deploy the exact reviewed commit to an isolated production-like environment.
2. Set `STOQUIFY_AGENT_RELEASE_ENVIRONMENT` to that environment's stable name.
3. Store `STOQUIFY_AGENT_RECONCILER_SECRET` in the deployment provider's managed secret store.
4. Use at least 32 random characters and prevent log interpolation.
5. Schedule the POST request every five minutes.
6. Prevent overlapping invocations or rely on the existing idempotent and lease-safe controls with a single-concurrency worker.
7. Set a request timeout below the schedule interval.
8. Retry transient network failures without bypassing the worker's own idempotency.
9. Alert on failed invocations, stale heartbeat, or repeated control-plane failures.
10. Prove three consecutive successful windows.
11. Prove 401 and 503 behavior.
12. Rotate through dual secret versions: deploy new, verify heartbeat, revoke old, verify old rejection.

The existing route uses a bearer secret. Workload identity would require a separately reviewed authentication adapter. Adding one is worthwhile only if the selected deployment provider supports it cleanly.

12. Alert Delivery, Acknowledgement, and Dead-Letter Closure

The current worker:

- Requires HTTPS in production
- Requires a strong secret in production

- Sends an idempotency key
- Uses a ten-second request timeout
- Uses capped exponential backoff with 80-100 percent jitter
- Claims work with lock recovery
- Stops after five attempts
- Persists terminal `DEAD_LETTER`
- Does not resurrect delivered or dead-letter deliveries on duplicate queue events
- Stores a bounded external request reference when supplied

Production-like closure procedure:

1. Configure `STOQUIFY_ASSURANCE_ALERT_WEBHOOK_URL` from managed configuration.
2. Configure `STOQUIFY_ASSURANCE_ALERT_WEBHOOK_SECRET` from the secret manager.
3. Create a controlled Workflow Assurance incident.
4. Verify webhook delivery and an external request reference.
5. Have the named incident owner acknowledge it within the defined SLO.
6. Link the acknowledgement to the Stoquify incident or event history.
7. Force a transient failure and verify retry timing.
8. Force five failures and verify terminal dead-letter.
9. Verify duplicate queueing does not change terminal state.
10. Allow the acknowledgement SLO to expire in a test and prove escalation to the backup owner.
11. Document the operator recovery procedure.

One residual repository gap remains: a dedicated authorized operator command for dead-letter replay or recovery is not yet demonstrated. Before a real pilot, either implement that protected operation with reason, idempotency, tenant scope, and audit, or define a manual database-free recovery path through the canonical Workflow Assurance service. Direct database editing is not acceptable.

13. Browser Certification Matrix

Scenario	Current result	Required before pilot
Authenticated desktop	Passed locally	Rerun from clean CI commit
Authenticated mobile	Passed locally	Rerun from clean CI commit
Keyboard generate	Passed locally	Retain in CI
Valid empty digest	Passed locally	Retain
Tenant evidence-link policy	Passed when links exist	Add explicit non-empty fixture
Feedback persistence	Passed locally	Bind allowed persistence counts in CI
Idempotent replay	Passed locally	Add concurrent duplicate case
Protected-data non-mutation	Passed locally	Rerun in production-like CI
No horizontal overflow	Passed locally	Retain
Kill switch	Existing focused test	Include in final certification bundle
Cross-tenant denial	Not certified in final bundle	Add
Unauthorized role	Not certified in final bundle	Add
Missing permission	Not certified in final bundle	Add
Module entitlement denial	Not certified in final bundle	Add
DRAFT/missing definition	Not certified in final bundle	Add
Manifest mismatch	Service/smoke coverage only	Add browser or API certification
Suspended release	Service/smoke coverage only	Add browser denial
Expired approval/owner coverage	Service coverage only	Add controlled certification
Stale heartbeat	PostgreSQL control coverage	Add end-to-end denial
Unhealthy alert transport	PostgreSQL control coverage	Add end-to-end denial
Empty/partial/stale/unavailable source	Partial	Add explicit fixtures and qualification assertions
Timeout/retry/concurrent duplicate	Partial	Add
Rollback takes effect	PostgreSQL suspension coverage	Add user-surface denial
Automated accessibility scan	Not run	Add axe or existing Stoquify accessibility harness

The final matrix should remain small and high-value. Reuse service tests for exhaustive state combinations and reserve Playwright for the critical authorization, user-visible, rollback, and data-integrity paths.

14. Sequenced Execution Roadmap

Stage A: Finish repository evidence

1. Add protected dead-letter recovery or an equivalent canonical recovery operation.
2. Complete the high-risk negative/degradation browser cases.
3. Add an explicit non-empty digest fixture for evidence-link certification.
4. Bind allowed persistence counts for agent run, feedback, Workflow Assurance, release, and audit records.
5. Add automated accessibility scanning.
6. Rerun all local gates.

Exit: repository evidence is complete and the worktree contains only reviewed intended changes.

Stage B: Complete security response

1. Inventory potentially exposed non-test credentials.

2. Rotate, redeploy, revoke, and verify.
3. Attach the rotation register to the security review.

Exit: no potentially exposed non-test credential remains valid.

Stage C: Create immutable source and CI evidence

1. Obtain authorization to commit.
2. Create and review the exact commit.
3. Run mandatory CI, including the full enabled-pilot bundle.
4. Retain immutable evidence and hashes.

Exit: one clean commit has a fully green, immutable certification bundle.

Stage D: Deploy production-like controls

1. Deploy the exact commit and migration set.
2. Configure managed reconciler and alert credentials.
3. Enable five-minute scheduling.
4. Prove heartbeat, authorization failure behavior, alert delivery, retry, dead-letter, acknowledgement, and escalation.
5. Complete secret rotation drill.

Exit: production-like reconciliation and alerting are healthy and owned.

Stage E: Record real governance

1. Record product decision.
2. Record independent security decision.
3. Assign and acknowledge all six responsibilities.
4. Confirm runbook, coverage, activation window, pilot organization, and roles.

Exit: governance records are valid and bound to the same manifest and commit.

Stage F: Independent release decision

1. Run `017-aqstoqflow-enterprise-release-gate`.
2. Verify no evidence is stale.
3. Issue `GO`, `CONDITIONAL GO`, or `NO-GO`.

Exit: a documented independent decision exists.

Stage G: Separately authorized activation

Only after an explicit separate authorization:

1. Activate one internal pilot organization through the canonical protected transition.
2. Monitor reconciliation, alert delivery, denial, failure, stale output, and run metrics.
3. Suspend immediately on a stop condition.
4. Do not begin Phase 3 until the controlled pilot exit criteria pass.

15. Rollback and Emergency Disable

Rollback must preserve evidence:

1. Use the kill switch for immediate universal denial.

2. Use the canonical suspend transition for the tenant package.
3. Verify the Command Agent surface denies execution.
4. Disable the scheduler only if the scheduler itself is unsafe; normally keep reconciliation running to prove recovery.
5. Revoke compromised credentials and deploy replacements.
6. Notify rollout, rollback, support, pilot, security-incident, and backup owners.
7. Preserve release, certification, audit, Workflow Assurance, alert, and fingerprint evidence.
8. Do not delete additive migrations merely to roll back application behavior.
9. Retire a release only after it is suspended and no longer needed for recovery.

Automatic blocking drift already routes through guarded suspension. Tenant suspension does not need to mutate the global agent definition.

16. RACI and Human Completion Register

Work item	Responsible	Accountable	Consulted	Informed	Current status
Credential rotation	Platform/security engineer	Security owner	Release owner	Product owner	Missing real evidence
Source review and commit	Engineering lead	Release owner	Security/QA	Product owner	Not authorized/executed
Product decision	Product approver	Product owner	Pilot/support	Release owner	Missing
Security decision	Security approver	Security owner	Platform/release	Product owner	Missing
Scheduler deployment	Platform engineer	SRE owner	Security	Support/pilot	Missing
Alert transport	SRE engineer	Security-incident owner	Platform/support	Product owner	Missing
Pilot certification	QA/release engineer	Release owner	Security/pilot	Support	Local only
Rollback drill	Rollback owner	Release owner	SRE/security	Support/pilot	Missing production-like proof
Pilot support coverage	Support owner	Pilot owner	Product/SRE	Pilot users	Missing
Enterprise gate	Independent reviewer	Release authority	Product/security/SRE	All owners	Blocked by dependencies

No entry may be completed with invented names, test identities, or inferred acknowledgements.

17. Residual Risks

1. The current certificate is local and based on a dirty worktree.
2. Real credential rotation is not proven.
3. Scheduler and alert transport are not deployed.
4. Owner acknowledgement and escalation are not proven.
5. The negative/degradation matrix is incomplete.
6. Dead-letter persistence is implemented, but authorized recovery is not demonstrated.
7. The non-empty evidence-link path should be made deterministic in certification.
8. Automated accessibility scanning is still absent from the final bundle.
9. Local E2E approval and owner fixtures could be misread as real governance unless reports keep the distinction explicit.

10. Phase 3 would multiply operational risk if started before the Phase 2 control plane is exercised in a real controlled pilot.

18. Final Gate Decision

Gate	Decision
Release-control architecture	Passed locally
Permanent no-business-authority boundary	Passed locally
Migration and PostgreSQL persistence	Passed locally
Retirement lifecycle	Passed locally
Alert dead-letter and jitter	Passed locally
Sanitized browser evidence	Passed locally
Protected-data non-mutation	Passed locally
Keyboard, feedback, and replay	Passed locally
Complete negative/degradation/accessibility matrix	Partial
Credential rotation	Blocked
Clean-commit CI certification	Blocked
Real product/security approval	Blocked
Real operational owners	Blocked
Deployed five-minute reconciliation	Blocked
Delivered/acknowledged/escalated production-like alert	Blocked
Independent enterprise release gate	Blocked
Controlled internal pilot	No-go
Phase 3	Not authorized

Current authorized status: [READY FOR CONTROLLED REMEDIATION VERIFICATION](#)

Next technically appropriate action: finish the remaining high-risk negative and recovery evidence, obtain authorization for a reviewed commit, rotate potentially exposed credentials, deploy the exact commit to an isolated production-like environment with managed scheduler and webhook credentials, record real approvers and owners, run the complete clean-commit certification bundle, and then invoke [017-aqstoqflow-enterprise-release-gate](#).

Activation remains a separate explicit decision.